

Ridge Regression

Importing required packages

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

pd.set_option('display.float_format', lambda x: '%.9f' % x)
```

Importing data

```
teams = pd.read_csv(r"D:\vj\programing study material\my portfolio
projects\4 Ridge Regression\Data\teams.csv")

teams
```

	team	year	athletes	events	age	height	weight	prev_medals
0	AFG	1964	8	8	22.0	161.0	64.2	0.0
1	AFG	1968	5	5	23.2	170.2	70.0	0.0
2	AFG	1972	8	8	29.0	168.3	63.8	0.0
3	AFG	1980	11	11	23.6	168.4	63.2	0.0
4	AFG	2004	5	5	18.6	170.8	64.8	0.0
...	...	...	...	...	...	...	...	...
2009	ZIM	2000	26	19	25.0	179.0	71.1	0.0
2010	ZIM	2004	14	11	25.1	177.8	70.5	0.0
2011	ZIM	2008	16	15	26.1	171.9	63.7	3.0
2012	ZIM	2012	9	8	27.3	174.4	65.2	4.0
2013	ZIM	2016	31	13	27.5	167.8	62.2	0.0

[2014 rows x 9 columns]

Creating Train and Test data

```
train, test = train_test_split(teams, test_size=0.2, random_state=1)
```

Assigning X(Predictors) and y(Target)

```
predictors = ["athletes", "events"]  
target = "medals"
```

To avoid the copy warning from pandas we use .copy() function

```
X = train[predictors].copy()  
y = train[[target]].copy()
```

X

	athletes	events
1322	6	6
1872	119	80
953	4	4
1117	2	2
1993	43	25
...	...	...
1791	40	25
1096	36	23
1932	719	245
235	13	11
1061	50	38

[1611 rows x 2 columns]

y

	medals
1322	0
1872	5
953	0
1117	0
1993	0
...	...
1791	1
1096	1
1932	264
235	0
1061	3

[1611 rows x 1 columns]

Calculating mean and Standard deviation for X

```
x_mean = X.mean()  
x_std = X.std()
```

x_mean

```

athletes    74.409063
events      35.990068
dtype: float64

x_std

athletes    127.250043
events      48.978737
dtype: float64

```

Creating the scalar data

```

X = (X - x_mean) / x_std

X.describe()

# We rescaled our STD to 1 and mean to Zero

```

	athletes	events
count	1.611000e+03	1.611000e+03
mean	-1.338677e-17	1.287852e-18
std	1.000000e+00	1.000000e+00
min	-5.768883e-01	-7.143930e-01
25%	-5.297371e-01	-6.123079e-01
50%	-4.197174e-01	-4.489717e-01
75%	-2.679027e-02	1.839560e-01
max	6.008571e+00	4.634867e+00

Creating an intercept column

```

X["intercept"] = 1
X = X[["intercept"] + predictors]

X

   intercept  athletes  events
1322         1 -0.537596 -0.612308
1872         1  0.350420  0.898552
953          1 -0.553313 -0.653142
1117         1 -0.569030 -0.693976
1993         1 -0.246829 -0.224384
...         ...      ...
1791         1 -0.270405 -0.224384
1096         1 -0.301839 -0.265219
1932         1  5.065546  4.267361
235          1 -0.482586 -0.510223
1061         1 -0.191820  0.041037

[1611 rows x 3 columns]

```

creating Alpha and Identity matrix

```
alpha = 2
I = np.identity(X.shape[1])

I
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Assigning top left value to 0, because we don't want to penalize the y-intercept

```
I[0][0] = 0

I
array([[0., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

creating penalty matrix

```
penalty = alpha * I
penalty
array([[0., 0., 0.],
       [0., 2., 0.],
       [0., 0., 2.]])
```

Implementin equation to solve of find B value

```
B = np.linalg.inv(X.T @ X + penalty) @ X.T @ y
B # value of B0, B1, B2

      medals
0  10.691496
1  61.857734
2 -34.632920
```

for clear understanding

```
B.index = ["intercept", "athletes", "events"]
B

      medals
intercept  10.691496
athletes   61.857734
events    -34.632920
```

following same process for test data

```
test_X = test[predictors]
test_X = (test_X - x_mean) / x_std
test_X["intercept"] = 1
test_X = test_X[["intercept"] + predictors]
```

```
test_X
```

	intercept	athletes	events
309	1	-0.553313	-0.653142
285	1	0.594035	1.000637
919	1	-0.144668	0.102288
120	1	0.146098	0.531045
585	1	-0.301839	-0.122299
...	...	...	...
541	1	-0.380425	-0.408138
1863	1	-0.191820	0.143122
622	1	-0.058224	0.388126
1070	1	-0.569030	-0.693976
1196	1	-0.553313	-0.653142

```
[403 rows x 3 columns]
```

Calculation predictions

```
predictions = test_X @ B
```

```
predictions
```

	medals
309	-0.914959
285	12.782156
919	-1.799893
120	1.337116
585	-3.744014
...	...
541	1.294285
1863	-6.130765
622	-6.352080
1070	-0.472980
1196	-0.914959

```
[403 rows x 1 columns]
```

Scikit learn Ridge regression

Comparing our Regression model with Scikit learn model

```
from sklearn.linear_model import Ridge
```

```
ridge = Ridge(alpha = alpha)
```

```

ridge.fit(X[predictors],y)
Ridge(alpha=2)
ridge.coef_
array([[ 61.85773366, -34.63292036]])
ridge.intercept_
array([10.69149597])

```

YES our manual Ridge equation and Scikit learn ridge model creates same value

Predicting using Scikit learn model

```

sklearn_predictions = ridge.predict(test_X[predictors])

```

comparing the manual prediction and scikit learn predictions

```

predictions - sklearn_predictions

      medals
309    0.000000000
285   -0.000000000
919   -0.000000000
120   -0.000000000
585   -0.000000000
...
541    0.000000000
1863  -0.000000000
622   -0.000000000
1070    0.000000000
1196    0.000000000

[403 rows x 1 columns]

```

Every row have a Zero value so our manual prediction are identical

Combining all the process together

```

def ridge_fit(train, predictors, target, alpha):
    X = train[predictors].copy()
    y = train[[target]].copy()

    x_mean = X.mean()
    x_std = X.std()

    X = (X - x_mean) / x_std
    X["intercept"] = 1

```

```

X = X[["intercept"] + predictors]

penalty = alpha * np.identity(X.shape[1])
penalty[0][0] = 0

B = np.linalg.inv(X.T @ X + penalty) @ X.T @ y
B.index = ["intercept", "athletes", "events"]
return B, x_mean, x_std

def ridge_predict(test, predictors, x_mean, x_std, B):
    test_X = test[predictors]
    test_X = (test_X - x_mean) / x_std
    test_X["intercept"] = 1
    test_X = test_X[["intercept"] + predictors]

    predictions = test_X @ B
    return predictions

from sklearn.metrics import mean_absolute_error

errors = []
alphas = [10**i for i in range(-2,4)]

for alpha in alphas:
    B, x_mean, x_std = ridge_fit(train, predictors, target, alpha)
    predictions = ridge_predict(test, predictors, x_mean, x_std, B)
    errors.append(mean_absolute_error(test[target], predictions))

alphas
[0.01, 0.1, 1, 10, 100, 1000]

errors
[6.3096408301611335,
 6.30604433195291,
 6.2722833764316235,
 6.114051204717768,
 7.156811236590455,
 6.9780545895757315]

```

alpha 10 gives the lowest error, At the same time if alpha goes high we got higher error